

Relatorio do trial

PARTICIPANTE

Luis Henrique Gonçalves Barbosa

KATA

Tarifador de Logística Urbana

TRATAMENTO

Com assistente de IA

TEMPO FINAL

00:52

TEMPO ESGOTADO

Nao

INICIO

16/09/2026, 13:06:19

FIM

16/09/2026, 13:07:11

RESULTADO E OBSERVACOES

8 de 8 testes unitários de aceitação passaram com sucesso (100%).

ARQUIVOS ANEXADOS

4 arquivo(s) de código ou texto anexado(s) ao trial.

- TarifadorLogistica.java (58 linhas)
- ModalidadeEntrega.java (18 linhas)
- TipoCarga.java (20 linhas)
- texto-do-trial.txt (58 linhas)

TarifadorLogistica.java

```
package br.ufc.lab02.logistica;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class TarifadorLogistica {

    private static final BigDecimal TARIFA_BASE_CURTA = BigDecimal.valueOf(10.00);
    private static final BigDecimal TARIFA_BASE_MEDIA = BigDecimal.valueOf(25.00);
    private static final BigDecimal VALOR_KM_EXCEDENTE = BigDecimal.valueOf(1.50);
    private static final BigDecimal VALOR_KG_EXCEDENTE = BigDecimal.valueOf(3.00);
    private static final BigDecimal FATOR_DESCONTO_FIDELIDADE = BigDecimal.valueOf(0.90);

    public BigDecimal calcularFrete(double distanciaKm, double pesoKg, ModalidadeEntrega
modalidade, TipoCarga tipoCarga, boolean clienteFidelidade) {
        if (distanciaKm <= 0) {
            throw new IllegalArgumentException("Distancia deve ser maior que zero.");
        }
        if (pesoKg <= 0 || pesoKg > 50.0) {
            throw new IllegalArgumentException("Peso deve ser maior que zero e de no
maximo 50 kg.");
        }
        if (modalidade == null || tipoCarga == null) {
            throw new IllegalArgumentException("Modalidade e tipo de carga sao
obrigatorios.");
        }

        // 1. Tarifa base por distância
        BigDecimal custoDistancia;
        if (distanciaKm <= 15.0) {
            custoDistancia = TARIFA_BASE_CURTA;
        } else if (distanciaKm <= 50.0) {
            custoDistancia = TARIFA_BASE_MEDIA;
        } else {
            double kmExcedente = distanciaKm - 50.0;
            custoDistancia =
TARIFA_BASE_MEDIA.add(VALOR_KM_EXCEDENTE.multiply(BigDecimal.valueOf(kmExcedente)));
        }

        // 2. Adicional por peso (acima de 5 kg)
        BigDecimal custoPeso = BigDecimal.ZERO;
        if (pesoKg > 5.0) {
            double kgExcedente = pesoKg - 5.0;
            custoPeso = VALOR_KG_EXCEDENTE.multiply(BigDecimal.valueOf(kgExcedente));
        }

        // 3. Subtotal e multiplicador de modalidade
        BigDecimal subtotalTransporte = custoDistancia.add(custoPeso);
        BigDecimal transporteComModalidade =
subtotalTransporte.multiply(BigDecimal.valueOf(modalidade.getFatorMultiplicador()));

        // 4. Taxa de manuseio especial
        BigDecimal totalComManuseio =
transporteComModalidade.add(tipoCarga.getTaxaManuseio());

        // 5. Desconto de fidelidade
        BigDecimal totalFinal = clienteFidelidade
            ? totalComManuseio.multiply(FATOR_DESCONTO_FIDELIDADE)
```

```
        : totalComManuseio;  
    return totalFinal.setScale(2, RoundingMode.HALF_EVEN);  
}  
}
```

ModalidadeEntrega.java

```
package br.ufc.lab02.logistica;

public enum ModalidadeEntrega {
    PADRAO(1.00),
    EXPRESSA(1.40),
    MESMO_DIA(1.80);

    private final double fatorMultiplicador;

    ModalidadeEntrega(double fatorMultiplicador) {
        this.fatorMultiplicador = fatorMultiplicador;
    }

    public double getFatorMultiplicador() {
        return fatorMultiplicador;
    }
}
```

TipoCarga.java

```
package br.ufc.lab02.logistica;

import java.math.BigDecimal;

public enum TipoCarga {
    NORMAL(BigDecimal.valueOf(0.00)),
    FRAGIL(BigDecimal.valueOf(15.00)),
    PERECIVEL(BigDecimal.valueOf(25.00));

    private final BigDecimal taxaManuseio;

    TipoCarga(BigDecimal taxaManuseio) {
        this.taxaManuseio = taxaManuseio;
    }

    public BigDecimal getTaxaManuseio() {
        return taxaManuseio;
    }
}
```

texto-do-trial.txt

```
package br.ufc.lab02.logistica;

import java.math.BigDecimal;
import java.math.RoundingMode;

public class TarifadorLogistica {

    private static final BigDecimal TARIFA_BASE_CURTA = BigDecimal.valueOf(10.00);
    private static final BigDecimal TARIFA_BASE_MEDIA = BigDecimal.valueOf(25.00);
    private static final BigDecimal VALOR_KM_EXCEDENTE = BigDecimal.valueOf(1.50);
    private static final BigDecimal VALOR_KG_EXCEDENTE = BigDecimal.valueOf(3.00);
    private static final BigDecimal FATOR_DESCONTO_FIDELIDADE = BigDecimal.valueOf(0.90);

    public BigDecimal calcularFrete(double distanciaKm, double pesoKg, ModalidadeEntrega
modalidade, TipoCarga tipoCarga, boolean clienteFidelidade) {
        if (distanciaKm <= 0) {
            throw new IllegalArgumentException("Distancia deve ser maior que zero.");
        }
        if (pesoKg <= 0 || pesoKg > 50.0) {
            throw new IllegalArgumentException("Peso deve ser maior que zero e de no
maximo 50 kg.");
        }
        if (modalidade == null || tipoCarga == null) {
            throw new IllegalArgumentException("Modalidade e tipo de carga sao
obrigatorios.");
        }

        // 1. Tarifa base por distância
        BigDecimal custoDistancia;
        if (distanciaKm <= 15.0) {
            custoDistancia = TARIFA_BASE_CURTA;
        } else if (distanciaKm <= 50.0) {
            custoDistancia = TARIFA_BASE_MEDIA;
        } else {
            double kmExcedente = distanciaKm - 50.0;
            custoDistancia =
TARIFA_BASE_MEDIA.add(VALOR_KM_EXCEDENTE.multiply(BigDecimal.valueOf(kmExcedente)));
        }

        // 2. Adicional por peso (acima de 5 kg)
        BigDecimal custoPeso = BigDecimal.ZERO;
        if (pesoKg > 5.0) {
            double kgExcedente = pesoKg - 5.0;
            custoPeso = VALOR_KG_EXCEDENTE.multiply(BigDecimal.valueOf(kgExcedente));
        }

        // 3. Subtotal e multiplicador de modalidade
        BigDecimal subtotalTransporte = custoDistancia.add(custoPeso);
        BigDecimal transporteComModalidade =
subtotalTransporte.multiply(BigDecimal.valueOf(modalidade.getFatorMultiplicador()));

        // 4. Taxa de manuseio especial
        BigDecimal totalComManuseio =
transporteComModalidade.add(tipoCarga.getTaxaManuseio());

        // 5. Desconto de fidelidade
        BigDecimal totalFinal = clienteFidelidade
            ? totalComManuseio.multiply(FATOR_DESCONTO_FIDELIDADE)
```

```
        : totalComManuseio;  
    return totalFinal.setScale(2, RoundingMode.HALF_EVEN);  
}  
}
```